RespFish

Tech Implementation

Recording respiration data

To read the (analog) respiration belt signal from the ADI device, we replace old Arduino with faster ESP - for example **ESP WROOM 02D** (https://www.reichelt.at/at/de/shop/produkt/entwicklungsboard_esp-wroom-02d-311743). It passes the data to Unity via serial (USB) or over a local network (UDP).
For the mobile phone app we will use the microphone as a breath sensor.

Visualising and processing respiration data

We will use **Unity3D** and **C#** for realtime visuals and data processing. With unity, we can build desktop, mobile and (if needed in the future) VR apps.

Main modules to be implemented

- `RespMeter`: measures respiration, returns a respiration level and an average respiration rate (in a defined window)
  - `BeltRespMeter:` uses respiration belt data received from ESP / ADI
  - `MicRespMeter:` uses microphone to estimate respiration level
- `RespStats`: used in `RespirationMeter` to calculate windowed averages
- `AsyncResp`: generates respiration signals that are not in sync with measured respiration
  - `AsyncRespSim`: Simulates respiration data, e.g. generatively using sum of sines or by modulating pre-recorded respiration data
  - `AsyncRespMod`: Modulates real-time respiration data using delays and predictions to
- `RespCurveDisplay:` visual representation of a respiration signal, either real-time or simulated or (real-time) modulated data.
- User interface / interaction: decision screens, score counters
- Recording utilities